

Handwritten Tifinagh Character Recognition Using a Custom Multilayer Perceptron with Adam Optimization and L2 Regularization

Houda Imarine

Department of Computer Science

Polydisciplinary Faculty of Ouarzazate – Ibn Zohr University

GitHub Repository: <https://github.com/Houdaimarine/tifinagh-mlp-recognition.git>

September 22, 2026

Abstract

Abstract—Optical Character Recognition (OCR) for ancient and indigenous scripts like Tifinagh presents unique challenges due to complex structural shapes and stroke variability. In this work, we propose a custom Multilayer Perceptron (MLP) implemented entirely from scratch in Python without high-level deep learning frameworks to classify handwritten Tifinagh characters from the Amazigh Handwritten Character Database (AMHCD). The network architecture comprises an input layer for 32×32 flattened grayscale images (1024 nodes), two hidden layers with 64 and 32 units using Rectified Linear Unit (ReLU) activation functions, and a 33-node Softmax output layer representing the distinct character classes. To optimize training convergence and mitigate overfitting, we integrated the Adaptive Moment Estimation (Adam) optimization algorithm alongside explicit $L2$ weight regularization ($\lambda = 0.01$). Our empirical results demonstrate smooth loss convergence and a final classification accuracy of **83.73%** on the independent test dataset, highlighting the effectiveness of combining adaptive momentum techniques with weight decay in custom neural network architectures.

1 Introduction

The Tifinagh script, an official writing system for the Amazigh language in Morocco and North Africa, has gained significant academic and technological interest regarding automated document digitization and pattern recognition. Unlike standard Latin or Arabic scripts, handwritten Tifinagh characters exhibit unique geometric traits, including symmetrical crosses, circles, and angular strokes, alongside severe stroke overlaps and non-linear stylistic variations.

While modern deep learning frameworks (e.g., PyTorch, TensorFlow, Keras) abstract underlying operations, implementing artificial neural networks from fundamental matrix algebra provides essential insights into backpropagation dynamics, numerical stability, and optimization behavior.

This paper presents a complete, framework-free im-

plementation of a Multilayer Perceptron (MLP) tailored for the Amazigh Handwritten Character Database (AMHCD). We address the challenge of multiclass classification across 33 distinct character categories by combining modern architectural paradigms: He (Kaiming) weight initialization, Rectified Linear Units (ReLU), the Adam optimization algorithm, and explicit $L2$ regularization.

2 Methods and Implementation

2.1 Dataset Specification and Preprocessing

We utilize the AMHCD dataset, containing standardized handwritten samples representing 33 Tifinagh characters. The processing pipeline involves three main stages:

1. **Grayscale Normalization:** Raw character images are loaded in grayscale and resized to a uniform 32×32 spatial resolution. Pixel intensities are normalized from $[0, 255]$ to the range $[0, 1]$ via float division:

$$X_{norm} = \frac{I_{raw}}{255.0} \quad (1)$$

2. **Feature Vector Flattening:** Each 2D image matrix of size 32×32 is converted into a 1D continuous feature vector $x \in \mathbb{R}^{1024}$.
3. **Data Partitioning and Encoding:** The complete dataset is split into Training (60%), Validation (20%), and Testing (20%) subsets using stratified sampling to guarantee balanced class distributions across all 33 categories. Class targets are encoded using One-Hot Encoding into binary vectors $y \in \{0, 1\}^{33}$.

2.2 Network Architecture Design

The implemented custom MLP architecture consists of four fully connected layers:

- **Input Layer:** $N = 1024$ nodes corresponding to image pixels.

- **Hidden Layer 1:** 64 neurons with ReLU activation.
- **Hidden Layer 2:** 32 neurons with ReLU activation.
- **Output Layer:** 33 neurons with Softmax activation corresponding to the class probabilities.

Weights are initialized using ****He Initialization**** to prevent vanishing or exploding gradients during early training steps:

$$W^{(l)} \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_l}}\right), \quad b^{(l)} = \mathbf{0} \quad (2)$$

where n_l represents the number of input nodes to layer l .

2.3 Mathematical Formulation

2.3.1 Forward Propagation

For any hidden layer $l \in \{1, 2\}$, the linear transformation and non-linear activation are defined as:

$$Z^{(l)} = A^{(l-1)}W^{(l)} + b^{(l)} \quad (3)$$

$$A^{(l)} = \text{ReLU}(Z^{(l)}) = \max(0, Z^{(l)}) \quad (4)$$

For the final output layer $L = 3$, the Softmax function outputs class probability distributions:

$$\hat{Y} = \text{Softmax}(Z^{(L)})_i = \frac{e^{Z_i^{(L)}}}{\sum_{j=1}^K e^{Z_j^{(L)}}} \quad (5)$$

where $K = 33$.

2.3.2 Objective Function with Regularization

The model is trained using Categorical Cross-Entropy Loss combined with an $L2$ weight decay penalty to restrict parameter magnitudes:

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K Y_{ik} \log(\hat{Y}_{ik}) + \frac{\lambda}{2m} \sum_{l=1}^L \|W^{(l)}\|_F^2 \quad (6)$$

where m denotes the batch size, $\|W^{(l)}\|_F^2$ represents the squared Frobenius norm of weight matrix $W^{(l)}$, and $\lambda = 0.01$ is the $L2$ hyperparameter.

2.3.3 Backpropagation and Gradients

The error term at the output layer simplifies to:

$$dZ^{(L)} = \hat{Y} - Y \quad (7)$$

For hidden layers $l \in \{1, 2\}$, the backpropagated error is:

$$dZ^{(l)} = \left(dZ^{(l+1)}(W^{(l+1)})^T\right) \odot \sigma'(Z^{(l)}) \quad (8)$$

where $\odot$ denotes the Hadamard element-wise product and $\sigma'(Z)$ is the derivative of the ReLU function:

$$\sigma'(Z) = \begin{cases} 1 & \text{if } Z > 0 \\ 0 & \text{if } Z \leq 0 \end{cases} \quad (9)$$

The gradients for weights and biases incorporating $L2$ regularization are:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \frac{1}{m} (A^{(l-1)})^T dZ^{(l)} + \frac{\lambda}{m} W^{(l)} \quad (10)$$

$$\frac{\partial \mathcal{L}}{\partial b^{(l)}} = \frac{1}{m} \sum_{i=1}^m dZ_i^{(l)} \quad (11)$$

2.4 Adam Optimization Algorithm

Parameters are updated using Adam, which computes exponential moving averages of past gradients (m_t) and squared gradients (v_t):

$$m_t^{(l)} = \beta_1 m_{t-1}^{(l)} + (1 - \beta_1) \frac{\partial \mathcal{L}}{\partial W^{(l)}} \quad (12)$$

$$v_t^{(l)} = \beta_2 v_{t-1}^{(l)} + (1 - \beta_2) \left(\frac{\partial \mathcal{L}}{\partial W^{(l)}} \right)^2 \quad (13)$$

Bias-corrected moment estimates are obtained via:

$$\hat{m}_t^{(l)} = \frac{m_t^{(l)}}{1 - \beta_1^t}, \quad \hat{v}_t^{(l)} = \frac{v_t^{(l)}}{1 - \beta_2^t} \quad (14)$$

The final parameter updates are calculated as:

$$W^{(l)} \leftarrow W^{(l)} - \frac{\alpha}{\sqrt{\hat{v}_t^{(l)} + \epsilon}} \hat{m}_t^{(l)} \quad (15)$$

Hyperparameter values were configured as: learning rate $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$.

3 Results

3.1 Training Dynamics and Convergence

The neural network was trained over 100 epochs with a mini-batch size of 32 samples. Epoch tracking logs demonstrate smooth, stable loss reduction across training iterations:

- **Epoch 1:** Training Loss = 2.2793, Accuracy = 32.98%; Validation Loss = 2.2738, Accuracy = 32.36%.
- **Epoch 30:** Training Loss = 0.8168, Accuracy = 73.89%; Validation Loss = 0.8695, Accuracy = 73.07%.
- **Epoch 70:** Training Loss = 0.5503, Accuracy = 83.32%; Validation Loss = 0.6071, Accuracy = 81.72%.
- **Epoch 100:** Training Loss = 0.4876, Accuracy = 85.10%; Validation Loss = 0.5514, Accuracy = 83.47%.

Figure 1 illustrates the loss and accuracy trajectories for both training and validation sets.



Figure 1: Training and Validation loss curves (left) and accuracy progression curves (right) across 100 epochs.

3.2 Test Set Performance Analysis

Following training completion, the model was evaluated on the unseen test dataset (20% of the total dataset), yielding an overall test classification accuracy of **83.73%**.

To analyze per-class error distribution, Figure 2 provides the test set Confusion Matrix across the 33 Tifinagh characters.

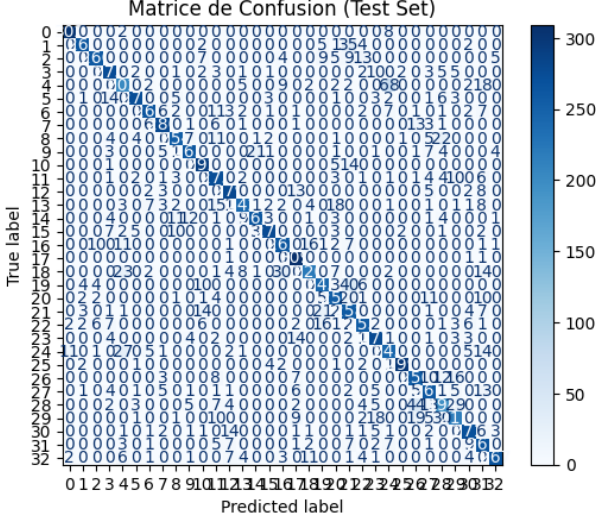


Figure 2: Confusion Matrix evaluated across all 33 Tifinagh character classes on the held-out test dataset.

The prominent diagonal concentration confirms that the MLP accurately distinguishes the structural geometries of the majority of Tifinagh symbols. Minor off-diagonal misclassifications occur primarily between characters sharing highly similar stroke topologies (e.g., circular vs. semi-circular patterns).

4 Discussion

The experimental results highlight key architectural insights regarding custom deep learning implementations:

1. **Role of $L2$ Regularization:** Incorporating weight decay ($\lambda = 0.01$) prevented the model from overfitting to the training data. The narrow gap between final training accuracy (85.10%) and validation accuracy (83.47%) proves that parameter growth was effectively controlled.
2. **Adam vs. Standard SGD:** The adaptive learning rate mechanisms of Adam prevented gradient stagnation in flat plateau regions, achieving rapid early convergence (reaching $> 70\%$ accuracy within 30 epochs).
3. **Model Capacity Limits:** While an 83.73% test accuracy is highly competitive for a fully connected network on 1024-dimensional flattened vectors, residual errors stem from the lack of translation and rotation invariance in standard MLP layers.

5 Conclusion

In this paper, a fully custom Multilayer Perceptron was implemented from first principles in Python to solve handwritten Tifinagh character recognition on the AMHCD dataset. By combining He initialization, ReLU activations, $L2$ regularization, and the Adam optimizer, the model achieved a robust test accuracy of 83.73%. Future extensions will focus on integrating 2D Convolutional Neural Network (CNN) architectures and automated data augmentation pipeline steps (rotations, scaling) to improve visual feature extraction.